TRABALHO 1 · INFRAESTRUTURA CLOUD AWS · UNESC

Passo a passo das configurações

CRUD de produtos e vendas · AWS EC2 + RDS PostgreSQL
Da versão Flask à API REST em Node.js com Swagger

1. Infraestrutura original na AWS (1ª etapa)
2. Projeto no computador (VS Code)
3. Git e GitHub
4. Migração no servidor EC2
5. Backup
6. Testes pelo Swagger
7. Manutenção e comandos úteis

Danielli Machado

Outubro de 2026

VISÃO GERAL

Como o sistema está hoje

Este documento registra, na ordem em que aconteceram, todas as configurações do trabalho: a montagem da infraestrutura na AWS, o projeto no computador, o versionamento no Git/GitHub, a troca da aplicação de Python/Flask para Node.js com Swagger, o backup e os testes.

Item	Valor
Endereço da API	http://56.124.25.34
Swagger (documentação e testes)	http://56.124.25.34/docs
Repositório	github.com/DanielliMachado/crud-aws (branch main)
Servidor	AWS EC2 crud-servidor · Ubuntu 26.04 · t3.micro · São Paulo (sa-east-1)
Banco de dados	AWS RDS crud-db · PostgreSQL 18 · db.t4g.micro · privado
Aplicação	Node.js 22 + Express + pg + swagger-ui-express
Pasta no servidor	/home/ubuntu/crud-aws
Serviço no servidor	crud.service (systemd), porta 8000
Pasta no computador	C:\Users\dani_\Desktop\crud-aws-main

Caminho de uma requisição

```
Cliente HTTP (Swagger / navegador / curl)
        |  porta 80
        v
EC2  ->  Nginx (proxy reverso)  ->  Node.js + Express (127.0.0.1:8000)
                                          |  porta 5432, SSL
                                          v
                                  RDS PostgreSQL (crud-db)
```

Caminho do código

```
Computador (VS Code)  --git push-->  GitHub  --git pull-->  EC2  --restart-->  no ar
```

Senha do banco: neste documento ela aparece sempre como ******. Ela fica só no servidor, dentro do arquivo do serviço, e nunca vai para o GitHub.

ETAPA 1 · 1ª ENTREGA

Infraestrutura original na AWS

Configuração montada na primeira etapa do trabalho, com a aplicação ainda em Python/Flask. Os valores abaixo são os que estão em uso; o passo a passo resume como cada peça foi criada no console da AWS.

1.1 Servidor EC2

1. Console AWS → região **América do Sul (São Paulo)** → **EC2** → **Executar instância**.
2. Nome: **crud-servidor**. Imagem: **Ubuntu Server 26.04 LTS**. Tipo: **t3.micro** (nível gratuito).
3. Par de chaves: criado um par RSA e baixado o arquivo **chave-crud.pem** (guardado em Downloads).
4. Armazenamento: volume **EBS gp3 de 8 GiB**.
5. Grupo de segurança (firewall) com duas regras de entrada:

Porta	Protocolo	Para quê	Origem
22	SSH	Administrar o servidor (só com a chave .pem)	Qualquer lugar*
80	HTTP	Acesso à aplicação	Qualquer lugar

* Em produção, o SSH seria restrito aos IPs conhecidos.

6. Instância iniciada. IP público: **56.124.25.34**.

1.2 Banco de dados RDS

1. Console AWS → **RDS** → **Criar banco de dados**.
2. Mecanismo: **PostgreSQL 18**. Modelo: **nível gratuito**. Identificador: **crud-db**. Usuário mestre: **postgres**.
3. Classe: **db.t4g.micro**. Armazenamento: **20 GiB**.
4. Conectividade: **Acesso público: Não**, conectado à mesma VPC da EC2.
5. Grupo de segurança do RDS: porta **5432** liberada **somente** para o grupo de segurança da EC2.
6. Backup automatizado habilitado (ver Etapa 5).
7. Após criado, anotado o endpoint: `crud-db.cdiuwqkci5ya.sa-east-1.rds.amazonaws.com`. Nome do banco usado pela aplicação: **crud**.

1.3 Stack original no servidor (versão Flask)

- Pacotes instalados com `apt`: Python, venv, Nginx e Git.
- Código baixado com `git clone` em `/home/ubuntu/crud-aws`; bibliotecas no `venv` pelo `requirements.txt`.
- Serviço `crud.service` no systemd rodando o **Gunicorn** na porta 8000, com as variáveis do banco.
- Nginx em `/etc/nginx/sites-available/crud` repassando a porta 80 para `127.0.0.1:8000`.
- Evoluções publicadas com o mesmo fluxo de deploy: campo **quantidade** (ALTER TABLE) e **vendas com vários itens** (tabelas vendas e itens_venda, com transação).

ETAPA 2 · 22/10/2026

Projeto no computador (VS Code)

Objetivo: transformar o projeto em um **backend** que só expõe endpoints HTTP (sem telas), com Swagger e sem autenticação, mantendo as mesmas tabelas e a mesma lógica.

2.1 Ponto de partida

- Pasta do projeto: `C:\Users\dani_\Desktop\crud-aws-main` (baixada como ZIP do GitHub, por isso ainda não estava ligada ao Git).
- Arquivos antigos: `app.py` (Flask com HTML), `requirements.txt`, `README.md`, `.gitignore`.

2.2 Estrutura nova do projeto

Arquivo	Função
`package.json`	Nome do projeto, dependências e comandos (npm start)
`src/server.js`	Inicia o Express, o Swagger em /docs, o /health e trata erros
`src/db.js`	Conexão com o PostgreSQL (SSL automático no RDS) e criação das tabelas
`src/routes/produtos.js`	CRUD de produtos com validação
`src/routes/vendas.js`	Histórico, resumo e registro de venda em transação
`src/swagger.js`	Documentação OpenAPI de todos os endpoints
`.env.example`	Modelo das variáveis de ambiente (sem senha real)
`.gitignore`	Ignora node_modules/, .env, venv/ e __pycache__/
`README.md`	Documentação do projeto e guia de migração na AWS

Foram removidos `app.py` e `requirements.txt` (continuam guardados no histórico do Git).

2.3 Tabelas (iguais às da versão Flask)

Criadas ao iniciar com `CREATE TABLE IF NOT EXISTS`: se já existem no RDS, nada é apagado.

- **produtos** (id, nome, preco, quantidade)
- **vendas** (id, data, valor_total)
- **itens_venda** (id, venda_id → vendas, produto_id → produtos, produto_nome, quantidade, preco_unitario)

2.4 Endpoints

Método e rota	O que faz	Respostas
`GET /produtos`	Lista todos os produtos	200
`GET /produtos/{id}`	Busca um produto	200, 404
`POST /produtos`	Cadastra {nome, preco, quantidade}	201, 400
`PUT /produtos/{id}`	Edita {nome, preco, quantidade}	200, 400, 404
`DELETE /produtos/{id}`	Exclui um produto	204, 404
`GET /vendas`	Histórico de vendas com itens	200
`GET /vendas/resumo`	Quantidade de vendas e total vendido	200
`GET /vendas/{id}`	Busca uma venda	200, 404
`POST /vendas`	{itens:[{produto_id, quantidade}]} – tudo ou nada	201, 400, 409
`GET /health`	Confere API e banco	200, 503
`GET /docs`	Swagger	200

2.5 Variáveis de ambiente

Variável	Valor no servidor	Descrição
`PORT`	8000	Porta da API
`DB_HOST`	endpoint do crud-db	Endereço do RDS
`DB_NAME`	crud	Nome do banco
`DB_USER`	postgres	Usuário
`DB_PASS`	******	Senha (só no servidor)
`DB_SSL`	automático	Liga sozinho quando o host é *.rds.amazonaws.com

2.6 Rodar no computador (opcional)

Pré-requisitos: **Node.js 18 ou maior** instalado (nodejs.org) e um PostgreSQL acessível. No terminal do VS Code (**Terminal** → **Novo terminal**), dentro da pasta do projeto:

```
npm install
copy .env.example .env      (depois edite o .env com os dados do banco)
npm start
```

Abra `http://localhost:3000/docs`. Para parar: **Ctrl + C** no terminal.

Como foi testado: o computador não tinha Node instalado. Foi usado um Node portátil e um PostgreSQL de teste em pasta temporária; todos os endpoints responderam o esperado, inclusive o 409 com o estoque voltando ao valor anterior.

ETAPA 3 · 22/10/2026

Git e GitHub

3.1 Ligar a pasta do computador ao repositório

A pasta tinha vindo de um ZIP, sem histórico. Para não mover nada de lugar e manter os commits antigos, foi feito no terminal, dentro de `crud-aws-main`:

```
git init
git remote add origin https://github.com/DanielliMachado/crud-aws.git
git fetch origin
git reset origin/main          (aponta para o histórico do GitHub, mantendo os arquivos novos)
git checkout -B main
git branch -u origin/main
```

Nome e e-mail do autor já estavam configurados no Git (Danielli Machado).

3.2 Commit e envio

```
git add .gitignore README.md app.py requirements.txt .env.example package.json package-lock.json src
git commit -m "Migra de Flask para API REST em Node.js com Swagger"
git push origin main
```

Resultado: commit `29c6faf` enviado para a branch **main**. Os PDFs da pasta não foram enviados (não faziam parte do repositório).

3.3 Histórico do repositório

Commit	Descrição
`81b43f2`	CRUD de produtos em Flask
`b9b56de`	alteração da tela
`9c35d47`	Adiciona campo quantidade
`82a7675`	Mostra quantidade na tabela
`642194c`	Pagina de vendas com varios produtos
`29c6faf`	Migra de Flask para API REST em Node.js com Swagger

3.4 Fluxo do dia a dia (a partir de agora)

No terminal do VS Code, dentro da pasta do projeto:

```
git add .
git commit -m "descreva a mudança"
git push
```

Depois, no servidor, atualizar conforme a Etapa 7.

ETAPA 4 · 22/10/2026

Migração no servidor EC2

Feita por partes, conferindo cada resultado antes de seguir. O site continuou no ar com o Flask até o momento da troca do serviço.

4.1 Entrar no servidor

Pelo console: **EC2** → **Instâncias** → **crud-servidor** → **Conectar** (EC2 Instance Connect). Ou pelo PowerShell do computador:

```
ssh -i "C:\Users\dani_\Downloads\chave-crud.pem" ubuntu@56.124.25.34
```

> O comando `ssh` é para rodar **no seu computador**. Se o terminal já mostra `ubuntu@ip-172-31-32-191:~$`, você já está dentro do servidor e não precisa dele.

4.2 Diagnóstico (só leitura)

```
cat /etc/os-release | head -2; whoami; pwd; ls
systemctl list-units --type=service --state=running | grep -iE "gunicorn|flask|crud|nginx"
sudo systemctl cat crud.service
sudo grep -rn proxy_pass /etc/nginx/
```

Descobertas: Ubuntu 26.04, usuário **ubuntu**, projeto em `/home/ubuntu/crud-aws`, serviço **crud.service** com o Gunicorn na porta 8000, e o Nginx (`/etc/nginx/sites-available/crud`) apontando para `127.0.0.1:8000`. Decisão: rodar o Node na **mesma porta 8000**, assim o Nginx não precisaria mudar.

4.3 Instalar o Node.js

```
sudo apt update && sudo apt install -y nodejs npm
node -v && npm -v
```

Resultado: Node **v22.22.1** e npm **9.2.0**.

4.4 Baixar o código novo e instalar dependências

```
cd ~/crud-aws && git status --short && git pull
npm install --omit=dev
```

O `git pull` trouxe o commit 29c6faf sem conflitos; o npm instalou 84 pacotes, 0 vulnerabilidades.

4.5 Teste com o banco real, sem tirar o site do ar

O Node foi ligado temporariamente na porta **3000**, usando as variáveis do próprio serviço (sem digitar a senha):

```
env $(sudo systemctl show crud.service -p Environment --value) PORT=3000 node src/server.js & \
  sleep 4; curl -s localhost:3000/health; curl -s localhost:3000/produtos; kill %1
```

Resultado: `{"status":"ok","banco":"ok"}` e a lista com os produtos já cadastrados.

4.6 Trocar o serviço de Gunicorn para Node

```
sudo sed -i \
  -e 's#^ExecStart=.*#ExecStart=/usr/bin/node src/server.js#' \
  -e 's#^Description=.*#Description=CRUD API Node#' \
  -e '/^Environment="DB_PASS/a Environment="PORT=8000"' \
  /etc/systemd/system/crud.service
```

Arquivo final `/etc/systemd/system/crud.service`:

```
[Unit]
Description=CRUD API Node
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/crud-aws
Environment="DB_HOST=crud-db.cdiuwqkci5ya.sa-east-1.rds.amazonaws.com"
Environment="DB_NAME=crud"
Environment="DB_USER=postgres"
Environment="DB_PASS=******"
Environment="PORT=8000"
ExecStart=/usr/bin/node src/server.js
Restart=always

[Install]
WantedBy=multi-user.target
```

4.7 Aplicar e conferir

```
sudo systemctl daemon-reload
sudo systemctl restart crud.service
sudo journalctl -u crud.service -n 5 --no-pager
```

Resultado esperado e obtido: `Started crud.service - CRUD API Node` e `API rodando na porta 8000 — Swagger em /docs`.

Conferência pelo endereço público: `/health` respondeu 200 com banco ok, `/docs` abriu e as 5 vendas antigas (R$ 2.457,60) continuaram lá.

4.8 Nginx

Nenhuma alteração. Ele já repassava a porta 80 para `127.0.0.1:8000`, onde agora roda o Node. Security Groups da EC2 e do RDS também não mudaram.

ETAPA 5 · 22 E 23/10/2026

Backup

Três camadas: dados (RDS), servidor (disco EBS da EC2) e código (GitHub).

5.1 Backup automático do RDS

1. Console → **RDS** → **Bancos de dados** → **crud-db** → aba **Manutenção e backups**.
2. Conferido: **Backups automatizados** habilitado; um snapshot automático por dia, às 00:39 (horário de Brasília).
3. Snapshots disponíveis na conferência: 21/10, 22/10 e 23/10 (o de 23/10 já contém os dados após a migração).
4. Snapshots automáticos mais antigos que o período de retenção são apagados pela própria AWS.

5.2 Snapshot manual do RDS

Existe o snapshot manual **crud-snapshot-apresentacao** (14/10/2026), que não expira. Para criar outro: **RDS** → **crud-db** → **Ações** → **Tirar snapshot**.

5.3 Snapshot do disco da EC2 (após a migração)

1. Console → **EC2** → **Elastic Block Store** → **Volumes**.
2. Marcado o volume `vol-0b7693a378c2c4dee` (8 GiB, gp3, anexado a crud-servidor).
3. **Ações** → **Criar snapshot**. Descrição: **crud-servidor-node-swagger** (tag opcional Name = crud-servidor-node).
4. **Criar snapshot** e acompanhar em **Elastic Block Store** → **Snapshots** até o status **Concluído**.

> A coluna *ID do snapshot* do volume (`snap-061cb22...`) é a imagem do Ubuntu que deu origem ao disco, não um backup.

5.4 Código

Cada commit no GitHub é um ponto de restauração; o servidor pode ser recriado com `git clone` + Etapa 4.

5.5 Manutenção pendente do RDS

Há uma atualização de sistema operacional disponível (*system-update*) e a janela de manutenção é 25/10/2026, 00:01–00:31. **Não aplicar antes da apresentação**: ela reinicia o banco e a API fica alguns minutos fora.

ETAPA 6

Testes pelo Swagger

Abrir `http://56.124.25.34/docs`. Em cada endpoint: clicar na barra → **Try it out** → preencher → **Execute** → ver **Code** e **Response body** em *Server response*.

Desligue a tradução automática do Chrome (ícone de tradução na barra de endereço → **Mostrar original** → **Nunca traduzir este site**). Senão GET vira "PEGAR", POST vira "PUBLICAR" etc. O campo **Servers** mostra só "/" (este servidor) e não precisa ser alterado.

#	Endpoint	Corpo / parâmetro	Esperado
1	GET /produtos	—	200 e a lista
2	POST /produtos	{"nome":"Produto Teste","preco":10.5,"quantidade":5}	201 – anotar o id
3	GET /produtos/{id}	id anotado	200
4	PUT /produtos/{id}	{"nome":"Produto Teste Editado","preco":12,"quantidade":5}	200
5	POST /vendas	{"itens":[{"produto_id":ID,"quantidade":2}]}	201, total 24; estoque 5 → 3
6	POST /vendas	{"itens":[{"produto_id":ID,"quantidade":99}]}	409; estoque continua 3
7	GET /vendas/resumo	—	total aumentou 24
8	POST /produtos	{"nome":"","preco":-1,"quantidade":1}	400
9	DELETE /produtos/{id}	id anotado	204; depois GET dá 404

Use **PUT** e **DELETE** apenas no produto de teste, para não alterar produtos reais.

ETAPA 7

Manutenção e comandos úteis

7.1 Publicar uma mudança de código

1. No computador (VS Code): `git add .` → `git commit -m "..."` → `git push`.
2. No servidor:

```
cd ~/crud-aws && git pull && npm install --omit=dev && sudo systemctl restart crud.service
```

7.2 Comandos no servidor

Comando	Para quê
`systemctl status crud.service --no-pager`	Ver se a API está rodando
`sudo journalctl -u crud.service -f`	Ver os logs ao vivo (Ctrl + C sai)
`sudo systemctl restart crud.service`	Reiniciar a API
`curl -s localhost:8000/health`	Testar API e banco de dentro do servidor
`sudo nginx -t && sudo systemctl reload nginx`	Validar e recarregar o Nginx

7.3 Problemas comuns

Sintoma	Causa provável	Solução
no pg_hba.conf entry ... no encryption	RDS exige SSL	Adicionar Environment="DB_SSL=true" no serviço
ETIMEDOUT / ECONNREFUSED no banco	Security Group do RDS	Liberar 5432 para o SG da EC2
password authentication failed	Senha errada no serviço	Conferir DB_PASS em crud.service
502 Bad Gateway no navegador	API parada ou porta errada	systemctl status crud.service; conferir PORT=8000
Página 404 em HTML (não JSON)	Serviço antigo ainda ativo	daemon-reload + restart (Etapa 4.7)
Identity file not accessible / hostname	ssh rodado com exemplo ou dentro do servidor	Usar o caminho real da chave, no PowerShell do PC

7.4 Pendências opcionais

- Trocar a senha do RDS (RDS → crud-db → Modificar → senha mestre) e atualizar `DB_PASS` no `crud.service` + daemon-reload + restart.
- Apagar o ambiente Python antigo: `rm -rf ~/crud-aws/venv`.
- Remover o campo "Servers" do Swagger e bloquear a tradução automática da página.
- Aplicar a atualização pendente do RDS depois da apresentação.